

PROGRAM 1

AIM - Write a program to perform addition, subtraction, multiplication, Division in C++ using Oop

Software used - Online C++ compiler

Algorithm –

1. Start.
2. Declare variables: 'a', 'b', 'sum', 'sub', 'mul', 'dv'.
3. Prompt the user to enter two numbers.
4. Input the numbers and store them in 'a' and 'b'.
5. Calculate the sum ('sum = a + b'), difference ('sub = a - b'), product ('mul = a * b'), and division ('dv = a / b').
6. Display the results of addition, subtraction, multiplication, and division.
7. End

Source code –

```
#include <iostream>

int main(){
    int a,b,sum,sub,mul,dv;

    std::cout << "enter the first number:";

    std::cin>>a;

    std::cout<<"enter the second number:";

    std::cin>>b;

    sum=a+b;

    sub=a-b;

    mul=a*b;

    dv=a/b;

    std::cout<<"the addition is "<<sum<<std::endl;

    std::cout<<"the subtraction is "<<sub<<std::endl;

    std::cout<<"the multiplication is "<<mul<<std::endl;

    std::cout<<"the division is "<<dv;

    return 0;
}
```

Output -

```
enter the first number:4545
enter the second number:45
the addition is 4590
the subtraction is 4500
the multiplication is 204525
the division is 101
```

PROGRAM 2

AIM - Using the concept of function overloading, write function for calculating area of triangle, circle and rectangle

Software used - Online C++ compiler

Algorithm –

1. Start the program and declare functions to calculate the area for triangle, circle, and rectangle.
2. Display prompt for choice and input 'c'.
3. If 'c == 1', input base 'b' and height 'h', then compute the area of the triangle.
4. Else if 'c == 2', input radius 'r', then compute the area of the circle.
5. Else if 'c == 3', input length 'l' and width 'w', then compute the area of the rectangle.
6. If none of the above, display "invalid choice."
7. End the program.

Source code –

```
#include <iostream>

int area(int b,int h){
    return 0.5*b*h;
}

int area(int r){
    return 3.1415*r*r;
}

double area(double l,double w){
    return l*w;
}

int main(){
    int c;

    std::cout<<"enter choice - ";

    std::cin>>c;

    if(c==1){
        int b,h;

        std::cout<<"enter the breath - ";

        std::cin>>b;

        std::cout<<"enter the height - ";

        std::cin>>h;
```

```

        std::cout<<"area of triangle is "<<area(b,h);
    }
    else if(c==2){
        int r;
        std::cout<<"enter radius - ";
        std::cin>>r;
        std::cout<<"area of circle is "<<area(r);
    }
    else if(c==3) {
        double l,w;
        std::cout<<"enter the lenght - ";
        std::cin>>l;
        std::cout<<"enter the weath - ";
        std::cin>>w;
        std::cout<<"area of rectangle is "<<area(l,w);
    }
    else{
        std::cout<<"invalid choice";
    }
    return 0;
}

```

Output -

```

enter choice - 1
enter the breath - 23
enter the height - 24
area of triangle is 276

```

```

enter choice - 2
enter radius - 34
area of circle is 3631

```

```

enter choice - 3
enter the lenght - 21
enter the weath - 12
area of rectangle is 252

```

PROGRAM 3

AIM – Write a program to perform multiplication of 2D metrics in C++ using Oop

Software used - Online C++ compiler

Algorithm -

1. Start Program:
2. Declare constants 'm = 3', 'n = 3', 'p = 3'.
3. Initialize matrices 'A[m][n]', 'B[n][p]', and 'C[m][p]' (with 'C' initialized to 0).
4. Display Matrices A and B:
5. Print elements of matrix 'A'.
6. Print elements of matrix 'B'.
7. Matrix Multiplication:
8. or each element 'C[i][j]', compute the sum of products of corresponding elements from row 'i' of 'A' and column 'j' of 'B'.
9. Display Resulting Matrix C:
10. Print elements of matrix 'C'.
11. End Program.

Source code –

```
#include <iostream>

using namespace std;

int main() {

    const int m = 3;

    const int n = 3;

    const int p = 3;

    int A[m][n] = {

        {1, 2, 3},

        {4, 5, 6},

        {2, 4, 6}    };

    int B[n][p] = {

        {7, 11, 5},

        {9, 10, 6},

        {3, 12, 4}  };

    int C[m][p] = {0};

    cout << "Matrix A:" << endl;

    for (int i = 0; i < m; i++) {

        for (int j = 0; j < n; j++) {
```

```

        cout << A[i][j] << " ";
    }
    cout << endl;
}
cout << "Matrix B:" << endl;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < p; j++) {
        cout << B[i][j] << " ";
    }
    cout << endl;
}
for (int i = 0; i < m; i++) {
    for (int j = 0; j < p; j++) {
        for (int k = 0; k < n; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
cout << "The resulting matrix C (A * B) is: " << endl;
for (int i = 0; i < m; i++) {
    for (int j = 0; j < p; j++) {
        cout << C[i][j] << " ";
    }
    cout << endl;
}
return 0;
}

```

Output –

```

Matrix A:
1 2 3
4 5 6
2 4 6
Matrix B:
7 11 5
9 10 6
3 12 4
The resulting matrix C (A * B) is:
34 67 29
91 166 74
68 134 58

```

PROGRAM 4

AIM – Write a program to create a class, which have data members as name, branch, roll no, age, sex, five subjects, display the name of the students & his percentage who has more than 70%, use array of objects in C++ using Oop

Software used - Online C++ compiler

Algorithm -

1. Start program and declare a 'Student' class with private attributes ('name', 'branch', 'rollNo', 'age', 'sex', 'marks [5] ').
2. Define constructor to initialize student details and marks array.
3. Implement 'calculatePercentage()' to compute and return the average of the marks.
4. Implement 'displayDetails()' to print the student's details.
5. Implement 'displayIfAbove70()' to calculate the percentage and display the student's details if the percentage is greater than 70%.
6. In the 'main () ' function:
 - Define marks arrays for each student.
 - Create an array of 'Student' objects with corresponding attributes.
7. Display all student details by looping through the array and calling 'displayDetails()'.
8. Display students with percentage above 70% by looping through the array and 'displayIfAbove70()'.
9. End program.

Source code –

```
#include<iostream>

using namespace std;

class Student {
    private:
        string name;
        string branch;
        int rollNo;
        int age;
        char sex;
        float marks[5];
    public:
        Student(string n, string b, int r, int a, char s, float m[]) {
            name = n;
            branch = b;
            rollNo = r;
            age = a;
```

```

        sex = s;

        for(int i = 0; i < 5; i++) {
            marks[i] = m[i];
        }
    }

    float calculatePercentage() {
        float total = 0;
        for(int i = 0; i < 5; i++) {
            total += marks[i];
        }
        return total / 5;
    }

    void displayDetails() {
        cout << "Name: " << name << ", "
            << "Branch: " << branch << ", "
            << "Roll No: " << rollNo << ", "
            << "Age: " << age << ", "
            << "Sex: " << sex << endl;
    }

    void displayIfAbove70() {
        float percentage = calculatePercentage();
        if(percentge > 70) {
            cout << "Name: " << name << ", "
                << "Branch: " << branch << ", "
                << "Roll No: " << rollNo << ", "
                << "Age: " << age << ", "
                << "Sex: " << sex << ", "
                << "Percentage: " << percentage << "%" << endl;
        }
    }
};

int main() {

```

```

float marks1[5] = {85, 90, 88, 75, 92};
float marks2[5] = {65, 70, 68, 72, 60};
float marks3[5] = {78, 80, 82, 85, 79};
float marks4[5] = {90, 95, 85, 88, 87};
float marks5[5] = {55, 60, 58, 62, 59};

Student students[5] = {
    Student("Kunal", "CSE", 101, 20, 'F', marks1),
    Student("Mayank", "ECE", 102, 21, 'M', marks2),
    Student("Aryan", "MECH", 103, 22, 'M', marks3),
    Student("Ishaan", "CIVIL", 104, 23, 'M', marks4),
    Student("Happy", "CSE", 105, 19, 'F', marks5)
};

cout << "Preview of All Students:" << endl;
for(int i = 0; i < 5; i++) {
    students[i].displayDetails();
}

cout << "\nStudents with percentage above 70%:" << endl;
for(int i = 0; i < 5; i++) {
    students[i].displayIfAbove70();
}

return 0;
}

```

Output –

```

Preview of All Students:
Name: Kunal, Branch: CSE, Roll No: 101, Age: 20, Sex: F
Name: Mayank, Branch: ECE, Roll No: 102, Age: 21, Sex: M
Name: Aryan, Branch: MECH, Roll No: 103, Age: 22, Sex: M
Name: Ishaan, Branch: CIVIL, Roll No: 104, Age: 23, Sex: M
Name: Happy, Branch: CSE, Roll No: 105, Age: 19, Sex: F

Students with percentage above 70%:
Name: Kunal, Branch: CSE, Roll No: 101, Age: 20, Sex: F, Percentage: 86%
Name: Aryan, Branch: MECH, Roll No: 103, Age: 22, Sex: M, Percentage: 80.8%
Name: Ishaan, Branch: CIVIL, Roll No: 104, Age: 23, Sex: M, Percentage: 89%

```

PROGRAM 5

AIM – Write a program to Create a class time with members hours, minutes and seconds. take input, add two-time objects and passing objects to function and display the result in C++ using Oop

Software used - Online C++ complier

Algorithm –

1. Start program and define a 'Time' class with private attributes ('hours', 'minutes', 'seconds'), and public methods ('inputTime()', 'displayTime()', and 'addTime()').
2. Define a constructor to initialize 'hours', 'minutes', and 'seconds' to 0.
3. implement 'inputTime()' to prompt the user for hours, minutes, and seconds and store them in the class attributes.
4. implement 'displayTime()' to display the time in 'HH:MM:SS' format, ensuring two digits for each field.
5. implement 'addTime()' to add the 'seconds', 'minutes', and 'hours' of two 'Time' objects, handling carryovers and ensuring the time format wraps correctly (seconds and minutes under 60, hours under 24).
6. In the 'main()' function, create three 'Time' objects: 't1', 't2', and 't3'.
7. input time for 't1' and 't2' using 'inputTime()' and calculate their sum by calling 't1.addTime(t2)' and storing the result in 't3'.
8. Display the sum of times using 't3.displayTime()'.
9. End program.

Source code –

```
#include<iostream>

using namespace std;

class Time {
    private:
        int hours;
        int minutes;
        int seconds;
    public:
        Time() {
            hours = 0;
            minutes = 0;
            seconds = 0;
        }
        void inputTime() {
            cout << "Enter hours: ";
            cin >> hours;
```

```

        cout << "Enter minutes: ";
        cin >> minutes;
        cout << "Enter seconds: ";
        cin >> seconds;
    }
    void displayTime() {
        cout << (hours < 10 ? "0" : "") << hours << ":"
            << (minutes < 10 ? "0" : "") << minutes << ":"
            << (seconds < 10 ? "0" : "") << seconds << endl;
    }
    Time addTime(Time t) {
        Time result;
        result.seconds = seconds + t.seconds;
        result.minutes = minutes + t.minutes + result.seconds / 60;
        result.seconds %= 60;
        result.hours = hours + t.hours + result.minutes / 60;
        result.minutes %= 60;
        result.hours %= 24;
        return result;
    }
};

int main() {
    Time t1, t2, t3;
    cout << "Enter the first time:" << endl;
    t1.inputTime();

    cout << "Enter the second time:" << endl;
    t2.inputTime();

    t3 = t1.addTime(t2);

    cout << "The sum of the two times is: ";

```

```
t3.displayTime();  
return 0;  
}
```

Output –

```
Enter the first time:  
Enter hours: 22  
Enter minutes: 30  
Enter seconds: 00  
Enter the second time:  
Enter hours: 8  
Enter minutes: 30  
Enter seconds: 00  
The sum of the two times is: 07:00:00
```

PROGRAM 6

AIM – Write a program to find biggest of 3 number using friend function in C++ using Oop

Software used - Online C++ compiler

Algorithm –

1. Start
2. Create a class named 'Number' that has three private integer data members ('a', 'b', 'c').
3. Implement a constructor for the 'Number' class to initialize 'a', 'b', and 'c' with given values.
4. Declare a friend function 'findLargest' inside the 'Number' class. This function will have access
5. to the private members of the 'Number' class.
6. Define the 'findLargest' function outside the class.
7. In this function, compare the three integers ('a', 'b', 'c') of the 'Number' instance and determine
8. the largest one.
9. In the 'main' function, create an instance of the 'Number' class with specific values.
10. Call the 'findLargest' function with the created 'Number' instance.
11. Print the result returned by 'findLargest'.
12. End

Source code –

```
#include <iostream>

using namespace std;

class Number {
private:
    int a, b, c;
public:
    Number(int x, int y, int z) : a(x), b(y), c(z) {}
    friend int findLargest(Number num);
};

int findLargest(Number num) {
    int largest = num.a;
    if (num.b > largest) {
        largest = num.b;
    }
    if (num.c > largest) {
        largest = num.c;
    }
}
```

```
    }  
    return largest;  
}  
int main() {  
    Number num(10, 20, 30);  
    int largest = findLargest(num);  
    cout << "The largest number is: " << largest << endl;  
    return 0;  
}
```

Output –

```
The largest number is: 30
```

```
=== Code Execution Successful ===|
```

PROGRAM 7

AIM – Write a program to find the greatest of two given numbers in two different classes using friend function in C++ using Oop

Software used - Online C++ compiler

Algorithm –

1. Start
2. Define 'ClassA' and 'ClassB' with private integer members
3. declare 'findGreatest' as a friend function.
4. Implement constructors in 'ClassA' and 'ClassB' to initialize their integer members.
5. Implement 'findGreatest' to compare
6. return the greater of the private integer members of 'ClassA' and 'ClassB'.
7. In 'main', read two integers from the user
8. create 'ClassA' and 'ClassB' objects with these
9. integers, call 'findGreatest',
10. print the result.
11. End

Source code –

```
#include <iostream>

using namespace std;

class ClassB;

class ClassA {
private:
    int numA;
public:
    ClassA(int a) : numA(a) {}

    friend int findGreatest(ClassA, ClassB);
};

class ClassB {
private:
    int numB;
public:
    ClassB(int b) : numB(b) {}
```

```

        friend int findGreatest(ClassA, ClassB);
};

int findGreatest(ClassA a, ClassB b) {
    if (a.numA > b.numB) {
        return a.numA;
    } else {
        return b.numB;
    }
}

int main() {
    int a, b;
    cout << "Enter the first number: ";
    cin >> a;
    ClassA objA(a);

    cout << "Enter the second number: ";
    cin >> b;
    ClassB objB(b);

    int greatest = findGreatest(objA, objB);
    cout << "The greatest number is: " << greatest << endl;
    return 0;
}

```

Output –

```

Enter the first number: 12
Enter the second number: 45
The greatest number is: 45

```

```

=== Code Execution Successful ===

```

PROGRAM 8

AIM – Write a program to Demonstrate the use of friend function with inlined assignment in C++ using Oop

Software used - Online C++ compiler

Algorithm –

1. Start
2. Define the 'Box' class.
3. Declare private members: 'length', 'width', and 'height'.
4. Define the constructor to initialize 'length', 'width', and 'height'.
5. Declare a friend function 'displayDimensions' that can access private members of 'Box'.
6. Implement the 'displayDimensions' function to print the dimensions of a 'Box' object.
7. In the 'main' function, create an instance of 'Box' with specified dimensions (e.g., length=10, width=5, height=8).
8. Call the 'displayDimensions' function with the created 'Box' object to display its dimensions.
9. End the program.

Source code –

```
#include <iostream>

using namespace std;

class Box {
private:
    int length;
    int width;
    int height;
public:
    Box(int l, int w, int h) : length(l), width(w), height(h) {}

    friend void displayDimensions(const Box& b) {
        cout << "Dimensions of the box:" << endl;
        cout << "Length: " << b.length << endl;
        cout << "Width: " << b.width << endl;
        cout << "Height: " << b.height << endl;
    }
};

int main() {
    Box myBox(10, 5, 8);
```

```
displayDimensions(myBox);  
return 0;  
}
```

Output –

```
Dimensions of the box:  
Length: 10  
Width: 5  
Height: 8  
  
=== Code Execution Successful ===|
```

PROGRAM 9

AIM – Write a program to find the sum of two numbers declared in a class and display the numbers and functions using friend class in C++ using Oop

Software used - Online C++ compiler

Algorithm –

1. Start
2. Declare 'Display' class.
3. Define 'Number' class with private members 'num1' and 'num2'
4. Declare 'Display' as a friend.
5. Define 'Display' class with functions 'findSum' and 'showDetails'.
6. In 'findSum', compute and return the sum of 'num1' and 'num2'.
7. In 'showDetails', print 'num1', 'num2', and their sum using 'findSum'.
8. In 'main', prompt the user for two integers and store them in 'a' and 'b'.
9. Create a 'Number' object with values 'a' and 'b'.
10. Create a 'Display' object.
11. Call 'showDetails' on the 'Display' object with the 'Number' object as an argument.
12. Return 0 to end the program.
13. End

Source code –

```
#include <iostream>

using namespace std;

class Display;

class Number {
private:
    int num1;
    int num2;
public:
    Number(int a, int b) : num1(a), num2(b) {}
    friend class Display;
};

class Display {
public:
    int findSum(const Number& n) {
        return n.num1 + n.num2;
    }
    void showDetails(const Number& n) {
```

```
        cout << "Number 1: " << n.num1 << endl;
        cout << "Number 2: " << n.num2 << endl;
        cout << "Sum: " << findSum(n) << endl;
    }
};

int main() {
    int a, b;
    cout << "Enter the first number: ";
    cin >> a;
    cout << "Enter the second number: ";
    cin >> b;

    Number num(a, b);
    Display display;
    display.showDetails(num);
    return 0;
}
```

Output –

```
Enter the first number: 21
Enter the second number: 24
Number 1: 21
Number 2: 24
Sum: 45

=== Code Execution Successful ===
```

PROGRAM 10

AIM – Write a program to perform addition of two complex numbers using constructor overloading. The first constructor which takes no argument is used to create objects which are not initialized, second which takes one argument is used to initialize real and imag parts to equal values and third which takes two argument is used to initialize real and imag to two different values in C++ using Oop

Software used - Online C++ compiler

Theory –

Constructor is a member function of a class that is used to initialize the objects of the class. Constructors do not have any return type and are automatically called when the object is created.

- The name of the constructor is the same as the class name
- No return type is there for constructors
- Called automatically when the object is created
- Always placed in the public scope of the class
- If a constructor is not created, the default constructor is created automatically and initializes the data member as zero
- Declaration of constructor name is case-sensitive
- A constructor is not implicitly inherited

Algorithm –

1. Start
2. Here's the algorithm with common steps combined for brevity:
3. Define a class named 'Complex' with two private floating-point variables: 'real' and 'imag'.
4. Create a default constructor that initializes 'real' and 'imag' to '0' and a constructor that initializes both to the same float value.
5. Create a constructor that initializes 'real' and 'imag' to two different float values.
6. Overload the '+' operator to add two 'Complex' objects, creating a new 'Complex' object for the result that stores the sum of the real and imaginary parts.
7. Create a method named 'display' that prints the complex number in the format "a + bi".
8. In the 'main' function, create instances of 'Complex' using the different constructors.
9. Display each complex number.
10. Add two complex numbers using the overloaded '+' operator and display the result.
11. Return '0' to indicate successful completion of the program.
12. Ends

Source code –

```
#include <iostream>

using namespace std;

class Complex {
    private:
        float real;
```

```

        float imag;
public:
    Complex() {
        real = 0;
        imag = 0;
    }
    Complex(float value) {
        real = value;
        imag = value;
    }
    Complex(float r, float i) {
        real = r;
        imag = i;
    }
    Complex operator+(const Complex& c) {
        Complex result;
        result.real = real + c.real;
        result.imag = imag + c.imag;
        return result;
    }
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1;
    Complex c2(5);
    Complex c3(3, 4);

    cout << "Complex number 1: ";
    c1.display();
    cout << "Complex number 2: ";

```

```
c2.display();  
cout << "Complex number 3: ";  
c3.display();  
Complex c4 = c2 + c3;  
cout << "Sum of Complex number 2 and Complex number 3: ";  
c4.display();  
return 0;  
}
```

Output –

```
Complex number 1: 0 + 0i  
Complex number 2: 5 + 5i  
Complex number 3: 3 + 4i  
Sum of Complex number 2 and Complex number 3: 8 + 9i  
  
=== Code Execution Successful ===
```

PROGRAM 11

AIM – Write a program to generate a Fibonacci series using copy constructor in OOPS using C++

Software used - Online C++ compiler

Theory –

A copy constructor is a type of constructor that initializes an object using another object of the same class. A constructor which creates an object by initializing it with an object of the same class, which has been created previously, is known as a copy constructor. The process of initializing members of an object through a copy constructor is known as copy initialization. It is also called member-wise initialization because the copy constructor initializes one object with the existing object, both belonging to the same class on a member-by-member copy basis.

Algorithm –

1. Start
2. Define a class named 'Fibonacci' with private members for a dynamic integer array 'series' and an integer 'size'.
3. Implement a constructor that initializes the Fibonacci series for a given number of terms, allocating memory for 'series' and calculating the Fibonacci numbers.
4. Implement a copy constructor that creates a deep copy of another 'Fibonacci' object by copying its series.
5. Implement a method named 'display' to print the Fibonacci series.
6. Implement a destructor to deallocate the memory for the 'series' array.
7. In the 'main' function, prompt the user for the number of terms in the Fibonacci series.
8. Create a 'Fibonacci' object with the specified number of terms and display the series.
9. Use the copy constructor to create a copy of the Fibonacci series and display the copied series.
10. Return '0' to indicate successful completion of the program.
11. End

Source code –

```
#include <iostream>

using namespace std;

class Fibonacci {
private:
    int* series;
    int size;
public:
    Fibonacci(int n) {
        size = n;
        series = new int[size];
        series[0] = 0;
```

```

        series[1] = 1;
        for (int i = 2; i < size; i++) {
            series[i] = series[i - 1] + series[i - 2];
        }
    }

    Fibonacci(const Fibonacci& other) {
        size = other.size;
        series = new int[size];
        for (int i = 0; i < size; i++) {
            series[i] = other.series[i];
        }
    }

    void display() {
        for (int i = 0; i < size; i++) {
            cout << series[i] << " ";
        }
        cout << endl;
    }

    ~Fibonacci() {
        delete[] series;
    }
};

int main() {
    int n;
    cout << "Enter the number of terms for Fibonacci series: ";
    cin >> n;
    Fibonacci fib(n);
    cout << "Fibonacci series: ";
    fib.display();
    Fibonacci fibCopy = fib;
    cout << "Copied Fibonacci series: ";
    fibCopy.display();
}

```

```
    return 0;  
}
```

Output –

```
Enter the number of terms for Fibonacci series: 8  
Fibonacci series: 0 1 1 2 3 5 8 13  
Copied Fibonacci series: 0 1 1 2 3 5 8 13  
  
=== Code Execution Successful ===
```

PROGRAM 12

AIM – Write a program to overload unary increment operator in OOps using C++

Software used - Online C++ compiler

Theory –

C++ comes with a large range of operators that can be used to perform mathematical and logical operations on numerical numbers. One such widely used operator is the unary operator. Unary Operators are employed in operations with a single operand. Unlike binary operators, unary operators do not need two operands to calculate the result. The unary operator is used to determine the new value of a single operand or variable. When using unary operators, the operand can be in either the prefix or postfix position. Unary operators have equal precedence, right-to-left associativity, and a variety of forms.

Algorithm –

1. Start
2. Initialize the class 'Counter' with a private member 'count' set to 0.
3. Define a constructor to initialize 'count' to 0.
4. Overload the prefix increment operator ('++') to increment 'count' and return the current object.
5. Overload the postfix increment operator ('++') to create a temporary copy, increment 'count', and return the copy.
6. Create a 'display()' function to print the current value of 'count'.
7. In 'main()', create an instance 'c' of class 'Counter'.
8. Use the prefix increment on 'c' and display the result.
9. Use the postfix increment on 'c' and display the result.
10. End

Source code –

```
#include <iostream>

using namespace std;

class Counter {
private:
    int count;
public:
    Counter() : count(0) {}
    Counter& operator++() {
        ++count;
        return *this;
    }
    Counter operator++(int) {
        Counter temp = *this;
```

```
        count++;  
        return temp;  
    }  
    void display() const {  
        cout << "Count: " << count << endl;  
    }  
};  
int main() {  
    Counter c;  
    ++c;  
    c.display();  
    c++;  
    c.display();  
    return 0;  
}
```

Output –

```
Count: 1  
Count: 2  
  
=== Code Execution Successful ===
```

PROGRAM 13

AIM – Write a program to overload unary increment operator in OOPS using C++

Software used - Online C++ compiler

Theory –

The new operator is mainly utilized to allocate memory for an object or an array of objects. It returns a pointer to the allocated memory. On the other hand, the delete operator is mainly utilized to deallocate memory that was previously allocated with new. It frees the memory pointed to by the pointer and sets the pointer to null.

Algorithm –

1. Start
2. Define a class
3. Overload the 'new' operator to allocate memory using 'malloc()' and handle allocation failure.
4. Overload the 'delete' operator to release memory using 'free()'.
5. Implement a constructor to initialize member variables.
6. Implement a destructor to perform cleanup if necessary.
7. In the 'main()' function, create an object of the class using the overloaded 'new' operator.
8. Delete the object using the overloaded 'delete' operator.
9. End

Source code –

```
#include <iostream>

using namespace std;

class MyClass {
public:
    int data;

    void* operator new(size_t size) {
        cout << "Custom new operator called. Allocating memory of size: " << size << " bytes"
        << endl;

        void* p = malloc(size);

        if (!p) {
            throw bad_alloc();
        }

        return p;
    }

    void operator delete(void* p) {
        cout << "Custom delete operator called. Releasing memory." << endl;
```

```
        free(p);
    }
    MyClass(int val) : data(val) {
        cout << "Constructor called. Data: " << data << endl;
    }
    ~MyClass() {
        cout << "Destructor called. Data: " << data << endl;
    }
};

int main() {
    MyClass* obj = new MyClass(42);
    delete obj;
    return 0;
}
```

Output –

```
Custom new operator called. Allocating memory of size: 4 bytes
Constructor called. Data: 42
Destructor called. Data: 42
Custom delete operator called. Releasing memory.

=== Code Execution Successful ===
```

PROGRAM 14

AIM – Write a program to create a base class basic info with data members name, roll no, sex and two member functions getdata and display. Derive a class physical fit from basic_info which has data members height and weight and member functions getdata display. Display all the information using object of derived class (Single Inheritance) in Oops using C++

Software used - Online C++ compiler

Theory –

The inheritance in which a single derived class is inherited from a single base class is known as the Single Inheritance. It is the simplest among all the types of inheritance since it does not include any kind of inheritance combination or different levels of inheritance. The child class can inherit all the members of the base class according to the visibility mode (i.e., private, protected, and public) that is specified during the inheritance. However, it is optional to define the visibility mode during the inheritance. If you do not define it, it sets the visibility mode to private by default.

Algorithm –

1. Start
2. Define class 'BasicInfo' with attributes 'name', 'rollNo', 'sex' and methods 'getData()' and 'display()'.
3. Define class 'PhysicalFit' that inherits from 'BasicInfo' with attributes 'height', 'weight' and methods 'getData()' and 'display()'.
4. In 'getData()' of 'BasicInfo', prompt for and input 'name', 'rollNo', and 'sex'.
5. In 'display()' of 'BasicInfo', print 'name', 'rollNo', and 'sex'.
6. In 'getData()' of 'PhysicalFit', call 'BasicInfo.getData()' and then prompt for and input 'height' and 'weight'.
7. In 'display()' of 'PhysicalFit', call 'BasicInfo.display()' and print 'height' and 'weight'.
8. In 'main()', create an instance of 'PhysicalFit' named 'pf'.
9. Call 'pf.getData()' to collect user input.
10. Print a blank line and "Displaying Information:".
11. Call 'pf.display()' to show the collected data.
12. End

Source code –

```
#include <iostream>

#include <string>

using namespace std;

class BasicInfo {
protected:
    string name;
    int rollNo;
    char sex;
```

```

public:
    void getData() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Roll No: ";
        cin >> rollNo;
        cout << "Enter Sex (M/F): ";
        cin >> sex;
    }
    void display() const {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Sex: " << sex << endl;
    }
};

class PhysicalFit : public BasicInfo {
private:
    float height;
    float weight;
public:
    void getData() {
        BasicInfo::getData();
        cout << "Enter Height (in cm): ";
        cin >> height;
        cout << "Enter Weight (in kg): ";
        cin >> weight;
    }
    void display() const {
        BasicInfo::display();
        cout << "Height: " << height << " cm" << endl;
        cout << "Weight: " << weight << " kg" << endl;
    }
}

```

```
};  
  
int main() {  
    PhysicalFit pf;  
    pf.getData();  
    cout << endl << "Displaying Information:" << endl;  
    pf.display();  
    return 0;  
}
```

Output –

```
Enter Name: kunal  
Enter Roll No: 23  
Enter Sex (M/F): m  
Enter Height (in cm): 90  
Enter Weight (in kg): 90  
  
Displaying Information:  
Name: kunal  
Roll No: 23  
Sex: m  
Height: 90 cm  
Weight: 90 kg
```

PROGRAM 15

AIM – Write a program to Create class first with data members book no, book name and member function getdata and putdata. Create class second with data members author name, publisher and members getdata and showdata. Derive a class third from first and second with data members no of pages and year of publication. Display all this information using array of objects of third class. (Multiple Inheritance) in OOPS using C++

Software used - Online C++ compiler

Theory –

Multiple Inheritance is the concept of the Inheritance in C++ that allows a child class to inherit properties or behaviour from multiple base classes. Therefore, we can say it is the process that enables a derived class to acquire member functions, properties, characteristics from more than one base class.

Algorithm –

1. Start
2. Create 'First' class with attributes 'bookNo' (integer) and 'bookName' (string). Implement methods:
3. 'getData()': Read book number and name from user input.
4. 'putData()': Display book number and name.
5. Create 'Second' class with attributes 'authorName' (string) and 'publisher' (string). Implement methods:
6. 'getData()': Read author name and publisher from user input.
7. 'showData()': Display author name and publisher.
8. Create 'Third' class that inherits from both 'First' and 'Second'. Add attributes 'noOfPages' (integer) and 'yearOfPublication' (integer). Implement methods:
9. 'getData()': Call 'getData()' from 'First' and 'Second', then read number of pages and year of publication.
10. 'showData()': Call 'putData()' from 'First', 'showData()' from 'Second', and display number of pages and year of publication.
11. Define a constant 'size' for the number of books and create an array of 'Third' objects named 'books'.
12. For each index from 0 to 'size - 1', prompt for details for book 'i + 1' and call 'getData()' on 'books[i]'.
13. Print a message indicating that the book details will be displayed.
14. For each index from 0 to 'size - 1', print the details of book 'i + 1' and call 'showData()' on 'books[i]'.
15. End

Source code –

```
#include <iostream>

#include <string>

using namespace std;

class First {
```

```
protected:
    int bookNo;
    string bookName;
public:
    void getData() {
        cout << "Enter Book Number: ";
        cin >> bookNo;
        cout << "Enter Book Name: ";
        cin.ignore();
        getline(cin, bookName);
    }
    void putData() const {
        cout << "Book Number: " << bookNo << endl;
        cout << "Book Name: " << bookName << endl;
    }
};

class Second {
protected:
    string authorName;
    string publisher;
public:
    void getData() {
        cout << "Enter Author Name: ";
        cin.ignore();
        getline(cin, authorName);
        cout << "Enter Publisher: ";
        getline(cin, publisher);
    }
    void showData() const {
        cout << "Author Name: " << authorName << endl;
        cout << "Publisher: " << publisher << endl;
    }
}
```

```

};

class Third: public First, public Second {
private:
    int noOfPages;
    int yearOfPublication;
public:
    void getData() {
        First::getData();
        Second::getData();
        cout << "Enter Number of Pages: ";
        cin >> noOfPages;
        cout << "Enter Year of Publication: ";
        cin >> yearOfPublication;
    }
    void showData() const {
        First::putData();
        Second::showData();
        cout << "Number of Pages: " << noOfPages << endl;
        cout << "Year of Publication: " << yearOfPublication << endl;
    }
};

int main() {
    const int size = 3;
    Third books[size];

    for (int i = 0; i < size; ++i) {
        cout << "Enter details for book " << (i + 1) << ":" << endl;
        books[i].getData();
    }
    cout << "\nBook Details:" << endl;
    for (int i = 0; i < size; ++i) {
        cout << "\nDetails of Book " << (i + 1) << ":" << endl;
    }
}

```

```
        books[i].showData();
    }
    return 0;
}
```

Output –

```
Enter details for book 1:
Enter Book Number: 1
Enter Book Name: gravity
Enter Author Name: newton
Enter Publisher: jk
Enter Number of Pages: 87
Enter Year of Publication: 2005
Enter details for book 2:
Enter Book Number: 2
Enter Book Name: why me
Enter Author Name: albert sharma
Enter Publisher: kj
Enter Number of Pages: 98
Enter Year of Publication: 2008
Enter details for book 3:
Enter Book Number: 3
Enter Book Name: ki
Enter Author Name: newson
Enter Publisher: jk
Enter Number of Pages: 99
Enter Year of Publication: 2012
```

Book Details:

```
Details of Book 1:
Book Number: 1
Book Name: gravity
Author Name: ewton
Publisher: jk
Number of Pages: 87
Year of Publication: 2005
```

```
Details of Book 2:
Book Number: 2
Book Name: why me
Author Name: lbert sharma
Publisher: kj
Number of Pages: 98
Year of Publication: 2008
```

```
Details of Book 3:
Book Number: 3
Book Name: ki
Author Name: ewson
Publisher: jk
Number of Pages: 99
Year of Publication: 2012
```

PROGRAM 16

AIM – Write a program to Design three classes STUDENT, EXAM, and RESULT. The STUDENT class has data members such as roll no, name. Create a class EXAM by inheriting the STUDENT class. The EXAM class adds data members representing the marks scored in six subjects. Derive the RESULT from the EXAM class and has its own data members such as total marks. WAP to model this relationship. (Multilevel Inheritance) in OOPS using C++

Software used - Online C++ compiler

Theory –

The inheritance in which a class can be derived from another derived class is known as Multilevel Inheritance. Suppose there are three classes A, B, and C. A is the base class that derives from class B. So, B is the derived class of A. Now, C is the class that is derived from class B. This makes class B, the base class for class C but is the derived class of class A. This scenario is known as the Multilevel Inheritance. The data members of each respective base class are accessed by their respective derived classes according to the specified visibility modes.

Algorithm –

1. Start
2. Define 'STUDENT' class with 'rollNo' and 'name'.
3. Implement method to input student details in 'STUDENT'.
4. Implement method to display student details in 'STUDENT'.
5. Define 'EXAM' class inheriting from 'STUDENT'.
6. Add an array for six subject marks in 'EXAM'.
7. Implement method to input exam marks in 'EXAM'.
8. Implement method to display exam marks in 'EXAM'.
9. Define 'RESULT' class inheriting from 'EXAM'.
10. Add a member for total marks in 'RESULT'.
11. Implement method to calculate total marks in 'RESULT'.
12. Implement method to display result details in 'RESULT'.
13. In 'main', create 'RESULT' object.
14. Call methods to input student and exam details.
15. Calculate total marks.
16. Display all information.
17. End

Source code –

```
#include <iostream>

#include <string>

using namespace std;

class STUDENT {
protected:
    int rollNo;
```

```

        string name;

public:
    void inputStudentDetails() {
        cout << "Enter Roll No: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin.ignore();
        getline(cin, name);
    }

    void displayStudentDetails() {
        cout << "Roll No: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};

class EXAM : public STUDENT {
protected:
    int marks[6];

public:
    void inputExamDetails() {
        cout << "Enter marks for 6 subjects:" << endl;
        for (int i = 0; i < 6; i++) {
            cout << "Subject " << (i + 1) << ": ";
            cin >> marks[i];
        }
    }

    void displayExamDetails() {
        cout << "Marks in Subjects:" << endl;
    }
};

```

```
        for (int i = 0; i < 6; i++) {  
            cout << "Subject " << (i + 1) << ": " << marks[i] << endl;  
        }  
    }  
};
```

```
class RESULT : public EXAM {  
private:  
    int totalMarks;  
  
public:  
    void calculateTotalMarks() {  
        totalMarks = 0;  
        for (int i = 0; i < 6; i++) {  
            totalMarks += marks[i];  
        }  
    }  
  
    void displayResult() {  
        displayStudentDetails();  
        displayExamDetails();  
        cout << "Total Marks: " << totalMarks << endl;  
    }  
};
```

```
int main() {  
    RESULT result;  
    result.inputStudentDetails();  
    result.inputExamDetails();  
    result.calculateTotalMarks();  
    result.displayResult();  
}
```

```
    return 0;  
}
```

Output –

```
Enter Roll No: 21  
Enter Name: Kunal  
Enter marks for 6 subjects:  
Subject 1: 45  
Subject 2: 47  
Subject 3: 36  
Subject 4: 39  
Subject 5: 40  
Subject 6: 45  
Roll No: 21  
Name: Kunal  
Marks in Subjects:  
Subject 1: 45  
Subject 2: 47  
Subject 3: 36  
Subject 4: 39  
Subject 5: 40  
Subject 6: 45  
Total Marks: 252
```

PROGRAM 17

AIM – Write a program to Create a base class called SHAPE. Use this class to store two double type values. Derive two specific classes called TRIANGLE and RECTANGLE from the base class. Add to the base class, a member function getdata to initialize base class data members and another member function display to compute and display the area of figures. Make display a virtual function and redefine this function in the derived classes to suit their requirements. Using these three classes design a program that will accept driven of a TRIANGLE or RECTANGLE interactively and display the area in OOPS using C++

Software used - Online C++ compiler

Theory –

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

Algorithm –

1. Start.
2. Define class 'SHAPE' with protected 'value1' and 'value2' and methods 'getdata()' and virtual 'display()'.
3. Define class 'TRIANGLE' that overrides 'display()' to calculate and print area as '0.5 * value1 * value2'.
4. Define class 'RECTANGLE' that overrides 'display()' to calculate and print area as 'value1 * value2'.
5. In 'main' function, declare 'SHAPE* shape' and an integer 'choice'.
6. Prompt user for shape choice (1 for Triangle, 2 for Rectangle).
7. If choice is 1, instantiate 'TRIANGLE' and assign to 'shape'; if choice is 2, instantiate 'RECTANGLE'.
8. If choice is invalid, print error message and terminate.
9. Call 'shape->getdata()' and then 'shape->display()'.
10. Delete 'shape' to free memory.
11. End.

Source code –

```
#include <iostream>

using namespace std;

class SHAPE {
protected:
    double value1;
    double value2;
```

```
public:
    void getdata() {
        cout << "Enter first value (base or width): ";
        cin >> value1;
        cout << "Enter second value (height or length): ";
        cin >> value2;
    }

    virtual void display() {
        cout << "Area: " << 0 << endl;
    }
};
```

```
class TRIANGLE : public SHAPE {
public:
    void display() override {
        double area = 0.5 * value1 * value2;
        cout << "Area of Triangle: " << area << endl;
    }
};
```

```
class RECTANGLE : public SHAPE {
public:
    void display() override {
        double area = value1 * value2;
        cout << "Area of Rectangle: " << area << endl;
    }
};
```

```
int main() {
    SHAPE* shape;
    int choice;
```

```
cout << "Choose shape (1 for Triangle, 2 for Rectangle): ";  
cin >> choice;
```

```
if (choice == 1) {  
    shape = new TRIANGLE();  
} else if (choice == 2) {  
    shape = new RECTANGLE();  
} else {  
    cout << "Invalid choice!" << endl;  
    return 1;  
}
```

```
shape->getdata();  
shape->display();
```

```
delete shape;  
return 0;  
}
```

Output –

```
Choose shape (1 for Triangle, 2 for Rectangle): 1  
Enter first value (base or width): 45  
Enter second value (height or length): 21  
Area of Triangle: 472.5  
  
=== Code Execution Successful ===
```

PROGRAM 18

AIM – Write a program to define the function template for calculating the square of given numbers with different data types in OOPS using C++

Software used - Online C++ compiler

Theory –

A template is a simple yet very powerful tool in C++. The simple idea is to pass the data type as a parameter so that we don't need to write the same code for different data types. For example, a software company may need to sort() for different data types. Rather than writing and maintaining multiple codes, we can write one sort() and pass the datatype as a parameter.

Algorithm –

1. Start
2. Define a template function `square` that takes a parameter of a generic type `T`.
3. Inside the function, return the result of `number \* number`.
4. Initialize an integer variable `int\_num` with a value (e.g., 5).
5. Call the `square` function with `int\_num` as the argument and print the result.
6. Initialize a float variable `float\_num` with a value (e.g., 4.5).
7. Call the `square` function with `float\_num` as the argument and print the result.
8. Initialize a double variable `double\_num` with a value (e.g., 7.2).
9. Call the `square` function with `double\_num` as the argument and print the result.
10. The program terminates after printing the square of each number.
11. End

Source code –

```
#include <iostream>

using namespace std;

template <typename T>
T square(T number) {
    return number * number;
}

int main() {
    int int_num = 5;
    cout << "Square of " << int_num << " (int): " << square(int_num) << endl;

    float float_num = 4.5f;
```

```
cout << "Square of " << float_num << " (float): " << square(float_num) << endl;

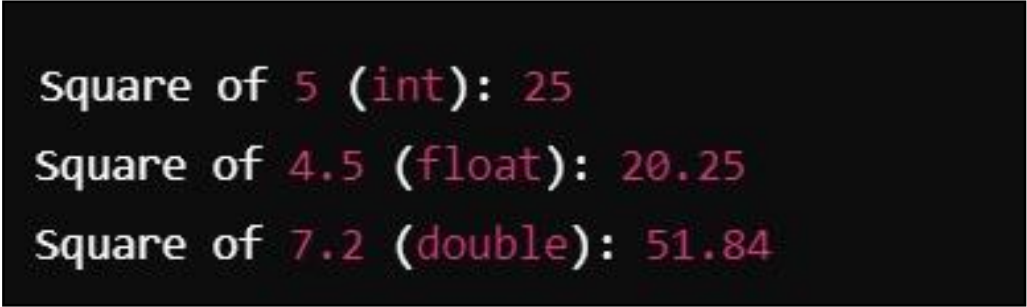
double double_num = 7.2;

cout << "Square of " << double_num << " (double): " << square(double_num) <<
endl;

return 0;

}
```

Output –



```
Square of 5 (int): 25
Square of 4.5 (float): 20.25
Square of 7.2 (double): 51.84
```

PROGRAM 19

AIM – Write a program to define the function template for swapping two items of various datatypes such as integers, float, and characters in OOPS using C++

Software used - Online C++ compiler

Theory –

A template is a simple yet very powerful tool in C++. The simple idea is to pass the data type as a parameter so that we don't need to write the same code for different data types. For example, a software company may need to sort() for different data types.

Algorithm –

1. Start
2. Define a template function `swapItems` to swap two values of type `T`.
3. In `main`, initialize two integers and print their values before swapping.
4. Call `swapItems` with the integer variables and print their values after swapping.
5. Initialize two floats and print their values before swapping.
6. Call `swapItems` with the float variables and print their values after swapping.
7. Initialize two characters and print their values before swapping.
8. Call `swapItems` with the character variables and print their values after swapping.
9. End

Source code –

```
#include <iostream>

using namespace std;

template <typename T>
void swapItems(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    cout << "Before swapping (int): x = " << x << ", y = " << y << endl;
    swapItems(x, y);
    cout << "After swapping (int): x = " << x << ", y = " << y << endl;
```

```
float p = 3.14f, q = 2.71f;
cout << "Before swapping (float): p = " << p << ", q = " << q << endl;
swapItems(p, q);
cout << "After swapping (float): p = " << p << ", q = " << q << endl;

char a = 'A', b = 'B';
cout << "Before swapping (char): a = " << a << ", b = " << b << endl;
swapItems(a, b);
cout << "After swapping (char): a = " << a << ", b = " << b << endl;
return 0;
}
```

Output –

```
Before swapping (int): x = 10, y = 20
After swapping (int): x = 20, y = 10
Before swapping (float): p = 3.14, q = 2.71
After swapping (float): p = 2.71, q = 3.14
Before swapping (char): a = A, b = B
After swapping (char): a = B, b = A
```

PROGRAM 20

AIM – Write a program to illustrate how Template function can be overloaded in Oops using C++

Software used - Online C++ compiler

Theory –

The name of the function templates are the same but called with different arguments is known as function template overloading. If the function template is with the ordinary template, the name of the function remains the same but the number of parameters differs. When a function template is overloaded with a non-template function, the function name remains the same but the function's arguments are unlike.

Algorithm –

1. Start
2. Create a template function `maximum` that takes two parameters of type `T`.
3. Inside the function, return the greater of the two values using the ternary operator.
4. Define a specialized version of the `maximum` function for `const char*`.
5. Inside this specialized function, use `strcmp` to compare the two strings.
6. Return the string that is lexicographically greater.
7. Initialize two integer variables `x` and `y` with values (e.g., 10 and 20).
8. Print the maximum value of `x` and `y` by calling the `maximum` function.
9. Initialize two double variables `p` and `q` with values (e.g., 3.14 and 2.71).
10. Print the maximum value of `p` and `q` by calling the `maximum` function.
11. Initialize two string literals `str1` and `str2` (e.g., "Apple" and "Banana").
12. Print the maximum string by calling the specialized `maximum` function.
13. The program terminates after printing the maximum values for all types.
14. End

Source code –

```
#include <iostream>
#include <cstring>
using namespace std;

template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}
```

```

template <D>
const char* maximum(const char* a, const char* b) {
    return (strcmp(a, b) > 0) ? a : b;
}

int main() {
    int x = 10, y = 20;
    cout << "Maximum (int): " << maximum(x, y) << endl;

    double p = 3.14, q = 2.71;
    cout << "Maximum (double): " << maximum(p, q) << endl;

    const char* str1 = "Apple";
    const char* str2 = "Banana";
    cout << "Maximum (string): " << maximum(str1, str2) << endl;

    return 0;
}

```

Output –

```

Maximum (int): 20
Maximum (double): 3.14
Maximum (string): Banana

=== Code Execution Successful ===

```

PROGRAM 21

AIM – Write a program to illustrate how to define and declare a class template for reading two data items from the keyboard and to find their sum in C++ using C++

Software used - Online C++ compiler

Theory –

The name of the function templates are the same but called with different arguments is known as function template overloading. If the function template is with the ordinary template, the name of the function remains the same but the number of parameters differs.

Algorithm –

1. Start
2. Define a template class Sum with private members a and b of type T.
3. Implement the readData function to input two values into a and b.
4. Implement the calculateSum function to return the sum of a and b.
5. In main, create an instance of Sum<int> and call readData and calculateSum.
6. Create an instance of Sum<float> and repeat steps 4 for floats.
7. Create an instance of Sum<double> and repeat steps 4 for doubles.
8. End the program.

Source code –

```
#include <iostream>

using namespace std;

template <typename T>
class Sum {
private:
    T a, b;
public:
    void readData() {
        cout << "Enter two values: ";
        cin >> a >> b;
    }
    T calculateSum() {
        return a + b;
    }
}
```

```
};  
  
int main() {  
    Sum<int> intSum;  
    intSum.readData();  
    cout << "Sum of integers: " << intSum.calculateSum() << endl;  
  
    Sum<float> floatSum;  
    floatSum.readData();  
    cout << "Sum of floats: " << floatSum.calculateSum() << endl;  
  
    Sum<double> doubleSum;  
    doubleSum.readData();  
    cout << "Sum of doubles: " << doubleSum.calculateSum() << endl;  
    return 0;  
}
```

Output –

```
Enter two values: 23 27  
Sum of integers: 50  
Enter two values: 45 5  
Sum of floats: 50  
Enter two values: 32 56  
Sum of doubles: 88  
  
=== Code Execution Successful ===
```

PROGRAM 22

AIM – Write a program to demonstrate the use of special functions, constructor, and destructor in the class template. The program is used to find the bigger of two entered numbers in OOPS using C++

Software used - Online C++ compiler

Theory –

A class template in C++ allows the creation of a class that can operate with any data type. It provides a way to define generic classes and functions, enabling code reuse and flexibility. In this program, the Compare class template is defined to accept a type parameter T, which can be replaced with any data type (e.g., int, float, double).

Algorithm –

1. Start
2. Declare two private member variables a and b of type T.
3. Implement a constructor that takes two parameters of type T (e.g., num1 and num2).
4. Initialize member variables a and b with the passed values.
5. Print a message indicating the constructor has been called and display the initialized values.
6. Create a public member function findBigger that compares a and b.
7. Return the larger value using the ternary operator.
8. Implement a destructor that prints a message indicating it has been called when the object is destroyed.
9. In the main function:
10. Create an instance of Bigger<int> with two integer values and call findBigger to print the larger integer.
11. Create an instance of Bigger<float> with two float values and call findBigger to print the larger float.
12. Create an instance of Bigger<double> with two double values and call findBigger to print the larger double.
13. The program terminates after printing the larger values for all data types and the destructors are called.
14. End

Source code –

```
#include <iostream>

using namespace std;

template <typename T>
class Bigger {
private:
    T a, b;
```

```

public:
    Bigger(T num1, T num2) {
        a = num1;
        b = num2;
        cout << "Constructor called. Values initialized: a = " << a << ", b = " << b <<
endl;
    }
    T findBigger() {
        return (a > b) ? a : b;
    }
    ~Bigger() {
        cout << "Destructor called. Cleaning up resources." << endl;
    }
};

int main() {
    Bigger<int> intObj(10, 20);
    cout << "The bigger number (int) is: " << intObj.findBigger() << endl;

    Bigger<float> floatObj(3.5f, 4.8f);
    cout << "The bigger number (float) is: " << floatObj.findBigger() << endl;

    Bigger<double> doubleObj(7.9, 5.4);
    cout << "The bigger number (double) is: " << doubleObj.findBigger() << endl;
    return 0;
}

```

Output –

```

Constructor called. Values initialized: a = 10, b = 20
The bigger number (int) is: 20
Constructor called. Values initialized: a = 3.5, b = 4.8
The bigger number (float) is: 4.8
Constructor called. Values initialized: a = 7.9, b = 5.4
The bigger number (double) is: 7.9
Destructor called. Cleaning up resources.
Destructor called. Cleaning up resources.
Destructor called. Cleaning up resources.

```

PROGRAM 23

AIM – Write a program to read a set of lines from the keyboard and to store it on a specified file in C++

Software used - Online C++ compiler

Theory –

The program demonstrates basic file handling in C++, including writing user input to a file. It uses `getline` to read entire lines of input, allowing for spaces within the input. Proper error handling is implemented to check if the file was opened successfully.

Algorithm –

1. Start
2. Include necessary headers for I/O, file handling, and strings.
3. Declare string variables `line` and `filename`.
4. Prompt user for the filename and read input.
5. Open an output file stream with the given filename and check for success.
6. Prompt user to enter lines of text and indicate to type "STOP" to end.
7. Clear the input buffer using `cin.ignore()`.
8. Start a loop to read user input until "STOP" is entered:
9. Read a line using `getline`.
10. If the line is "STOP," break the loop.
11. Write the line to the output file.
12. Print a message indicating completion and close the file.
13. Return 0 to indicate successful program execution.
14. End

Source code –

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    string line;
    string filename;
```

```

    cout << "Enter the filename to store the lines: ";
    cin >> filename;
    ofstream outFile(filename);

    if (!outFile) {
        cout << "Error opening file for writing." << endl;
        return 1;
    }
    cout << "Enter lines of text (type 'STOP' to end):" << endl;

    cin.ignore();
    while (true) {
        getline(cin, line);
        if (line == "STOP") {
            break;
        }
        outFile << line << endl;
    }
    cout << "Lines have been written to the file: " << filename << endl;
    outFile.close();
    return 0;
}

```

Output –

```

Enter the filename to store the lines: output.txt
Enter lines of text (type 'STOP' to end):
Hello, this is a test line.
This is another line.
STOP
Lines have been written to the file: output.txt

```

PROGRAM 24

AIM – Write a program to perform the deletion of white spaces such as horizontal tab, vertical tab, space, line feed, new line and carriage return from a text file and store the contents of the file without the white spaces on another file in C++ using C++

Software used - Online C++ compiler

Theory –

- The program demonstrates file handling in C++, including reading from and writing to files.
- It utilizes character classification functions from the `<cctype>` header to filter out whitespace.
- Proper error handling is implemented to ensure files open successfully before processing..

Algorithm –

1. Start
2. Include necessary headers for I/O, file handling, and character classification.
3. Declare string variables `inputFilename` and `outputFilename`, and a character variable `ch`.
4. Prompt user for the input filename and read it.
5. Prompt user for the output filename and read it.
6. Open the input file for reading and check for success.
7. Open the output file for writing and check for success.
8. Start a loop to read characters from the input file:
9. Read a character using `inputFile.get(ch)`.
10. If the character is not a whitespace (`!isspace(ch)`), write it to the output file.
11. Print a message indicating whitespace removal and that content is written to the output file.
12. Close both the input and output files.
13. Return 0 to indicate successful program execution..
14. End

Source code –

```
#include <iostream>
#include <fstream>
#include <cctype>
using namespace std;

int main() {
    string inputFilename, outputFilename;
```

```
char ch;

cout << "Enter the input filename: ";
cin >> inputFilename;
cout << "Enter the output filename: ";
cin >> outputFilename;

ifstream inputFile(inputFilename);
if (!inputFile) {
    cout << "Error opening input file!" << endl;
    return 1;
}
ofstream outputFile(outputFilename);
if (!outputFile) {
    cout << "Error opening output file!" << endl;
    return 1;
}
while (inputFile.get(ch)) {
    if (!isspace(ch)) {
        outputFile.put(ch);
    }
}

cout << "White spaces have been removed and the content has been written to: " <<
outputFilename << endl;

inputFile.close();
outputFile.close();
return 0;
}
```

Output –

```
Enter the input filename: input.txt
Enter the output filename: output.txt
white spaces have been removed and the content has been written to: output.txt
```

PROGRAM 25

AIM – Write a program to read a text file and display its contents on the screen in C++ using OOPS

Software used - Online C++ compiler

Theory –

- The program demonstrates basic file reading in C++, allowing users to input the name of a file and display its contents.
- It uses the `getline` function to read each line, which allows handling of spaces within the lines.
- Proper error handling is included to ensure the file opens successfully before attempting to read from it.

Algorithm –

1. Start
2. Include headers for input/output streams, file handling, and string operations.
3. Declare a string variable `filename` to store the name of the input file.
4. Declare a string variable `line` to store each line read from the file.
5. Display a message asking the user to enter the filename to read.
6. Read the filename input from the user.
7. Create an input file stream (`ifstream`) object using the provided filename.
8. Check if the file opened successfully.
9. If not, display an error message and return a non-zero value to indicate failure.
10. Start a while loop to read lines from the input file using `getline(inputFile, line)`.
11. Inside the loop, print each line to the console using `cout`.
12. After all lines have been read and printed, close the input file.
13. Return 0 to indicate successful completion of the program.
14. End

Source code –

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    string filename, line;
```

```
cout << "Enter the filename to read: ";  
cin >> filename;  
  
ifstream inputFile(filename);  
if (!inputFile) {  
    cout << "Error opening the file!" << endl;  
    return 1;  
}  
while (getline(inputFile, line)) {  
    cout << line << endl;  
}  
inputFile.close();  
return 0;  
}
```

Output –

```
Enter the filename to read: example.txt  
This is the first line.  
This is the second line.  
This is the third line.
```

PROGRAM 26

AIM – Write a program to copy the contents of a file into another in C++ using C++

Software used - Online C++ compiler

Theory –

- The program demonstrates basic file copying functionality in C++, allowing users to specify the input and output files.
- It reads the contents character by character, making it straightforward and effective for simple file operations.
- Proper error handling ensures that the program provides feedback if any files cannot be opened.

Algorithm –

1. Start
2. Include necessary headers for I/O and file handling.
3. Declare string variables inputFile and outputFile, and a character variable ch.
4. Prompt the user for the input filename and read it.
5. Prompt the user for the output filename and read it.
6. Open the input file for reading and check for success.
7. Open the output file for writing and check for success.
8. Start a loop to read characters from the input file:
9. Read a character using inputFile.get(ch).
10. Write the character to the output file using outputFile.put(ch).
11. Print a completion message indicating the contents have been copied.
12. Close both the input and output files.
13. Return 0 to indicate successful program execution.
14. End

Source code –

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    string inputFile, outputFile;
```

```

char ch;

cout << "Enter the input filename: ";
cin >> inputFilename;
cout << "Enter the output filename: ";
cin >> outputFilename;

ifstream inputFile(inputFilename, ios::in);
if (!inputFile) {
    cout << "Error opening input file!" << endl;
    return 1;
}
ofstream outputFile(outputFilename, ios::out);
if (!outputFile) {
    cout << "Error opening output file!" << endl;
    return 1;
}
while (inputFile.get(ch)) {
    outputFile.put(ch);
}

cout << "Contents copied from " << inputFilename << " to " << outputFilename <<
endl;

inputFile.close();
outputFile.close();
return 0;
}

```

Output –

```

Enter the input filename: source.txt
Enter the output filename: destination.txt
Contents copied from source.txt to destination.txt

```

PROGRAM 27

AIM – Write a program to read the class object of student info such as name, age, sex, height and weight from the keyboard and to store them on a specified file using read() and write functions. Again, the same file is opened for reading and displaying the contents of the file on the screen in C++ using C++

Software used - Online C++ compiler

Theory –

- The program demonstrates file I/O operations in C++ by writing student data to a file and then reading from it.
- It encapsulates student data and operations within a class, adhering to object-oriented programming principles.
- Proper error handling ensures that the program provides feedback if any files cannot be opened.

Algorithm –

1. Start
2. Include necessary headers for I/O and file handling.
3. Define a Student class with private attributes and public methods for reading, writing, and displaying data.
4. In the main function, declare a string variable for the filename and open an output file for writing.
5. Create an array of Student objects and input their details using readData().
6. Write each student's data to the file using writeData().
7. Close the output file and open the same file for reading.
8. Create a temporary Student object to read and display data from the file.
9. While reading from the file, use readData() and display data with displayData().
10. Close the input file and return 0 to indicate successful execution.
11. End

Source code –

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Student {
private:
```

```
    string name;

    int age;

    string sex;

    float height;

    float weight;

public:

    void readData() {

        cout << "Enter name: ";

        cin >> name;

        cout << "Enter age: ";

        cin >> age;

        cout << "Enter sex: ";

        cin >> sex;

        cout << "Enter height: ";

        cin >> height;

        cout << "Enter weight: ";

        cin >> weight;

    }

    void writeData(ofstream& outFile) {

        outFile << name << endl;

        outFile << age << endl;

        outFile << sex << endl;

        outFile << height << endl;

        outFile << weight << endl;

    }

    void readData(istream& inFile) {

        inFile >> name >> age >> sex >> height >> weight;

    }

    void displayData() {

        cout << "Name: " << name << ", Age: " << age << ", Sex: " << sex << ", Height: " << height << " cm, Weight: " << weight << " kg" << endl;
```

```
    }  
};  
  
int main() {  
    string filename = "student_info.txt";  
  
    ofstream outFile(filename);  
    if (!outFile) {  
        cout << "Error opening file for writing!" << endl;  
        return 1;  
    }  
    Student students[5];  
  
    students[0].readData(); // Aryan  
    students[1].readData(); // Kunal  
    students[2].readData(); // Shubham  
    students[3].readData(); // Suhani  
    students[4].readData(); // Priya  
  
    for (int i = 0; i < 5; i++) {  
        students[i].writeData(outFile);  
    }  
    outFile.close();  
  
    ifstream inFile(filename);  
    if (!inFile) {  
        cout << "Error opening file for reading!" << endl;  
        return 1;  
    }  
    cout << "\nStudent Info from the file:" << endl;  
    Student tempStudent;  
    while (inFile) {  
        tempStudent.readData(inFile);
```

```
        if (inFile) {  
            tempStudent.displayData();  
        }  
    }  
    inFile.close();  
    return 0;  
}
```

Output –

Student Info from the file:

Name: Aryan, Age: 20, Sex: male, Height: 175 cm, Weight: 70 kg

Name: Kunal, Age: 22, Sex: male, Height: 180 cm, Weight: 75 kg

Name: Shubham, Age: 21, Sex: male, Height: 172 cm, Weight: 68 kg

Name: Suhani, Age: 19, Sex: female, Height: 160 cm, Weight: 50 kg

Name: Priya, Age: 20, Sex: female, Height: 165 cm, Weight: 55 kg

PROGRAM 28

AIM – Write a program to read two numbers and then divide first no by second no and raise an exception if second number is zero in OOps using C++

Software used - Online C++ compiler

Algorithm –

1. Start
2. Include <stdexcept> for standard exceptions.
3. Create a class DivisionByZeroException that inherits from exception.
4. Override the what() method to return a custom error message.
5. Create a function double divide(double num1, double num2):
6. If num2 is zero, throw DivisionByZeroException.
7. Otherwise, return the result of num1 / num2.
8. Declare two double variables for user input.
9. Prompt the user for the first number and read it.
10. Prompt the user for the second number and read it.
11. Use a try block to:
12. Call divide(num1, num2) and store the result., Print the result.
13. Use a catch block to handle DivisionByZeroException:
14. Print the error message obtained from the what() method.
15. Return 0 to indicate successful completion.
16. End

Source code –

```
#include <iostream>
#include <stdexcept>
using namespace std;

class DivisionByZeroException : public exception {
public:
    const char* what() const noexcept override {
        return "Error: Division by zero is not allowed!";
    }
};
```

```
double divide(double num1, double num2) {  
    if (num2 == 0) {  
        throw DivisionByZeroException();  
    }  
    return num1 / num2;  
}  
  
int main() {  
    double num1, num2;  
  
    cout << "Enter the first number: ";  
    cin >> num1;  
    cout << "Enter the second number: ";  
    cin >> num2;  
    try {  
        double result = divide(num1, num2);  
        cout << "The result of division is: " << result << endl;  
    } catch (const DivisionByZeroException& e) {  
        cout << e.what() << endl;  
    }  
    return 0;  
}
```

Output –

```
Enter the first number: 10  
Enter the second number: 0  
Error: Division by zero is not allowed!
```

```
Enter the first number: 10  
Enter the second number: 5  
The result of division is: 2
```

PROGRAM 29

AIM – Write a program to raise an exception if any attempt is made to refer to an element whose index is beyond the array size in OOPS using C++

Software used - Online C++ compiler

Theory –

- Exception handling is a mechanism to manage errors or unusual conditions that occur during program execution.
- In C++, the try, catch, and throw keywords are used to implement exception handling.
- When an exception is thrown, control is transferred to the nearest matching catch block, which can handle the exception.

Algorithm –

1. Start
2. Include <iostream> and <stdexcept>.
3. Define IndexOutOfBoundsException class inheriting from exception with a custom what() method.
4. Create Array class with a private array of size 5.
5. Implement setElement(int index, int value) to set array values with bounds checking.
6. Implement getElement(int index) to retrieve array values with bounds checking.
7. In main(), create an Array object.
8. Use a try block to set a valid and an invalid element, catching exceptions.
9. Use another try block to get a valid and an invalid element, catching exceptions.
10. Return 0 to indicate successful execution.
11. End

Source code –

```
#include <iostream>
#include <stdexcept>
using namespace std;

class IndexOutOfBoundsException : public exception {
public:
    const char* what() const noexcept override {
        return "Error: Index is out of bounds!";
    }
}
```

```

};

class Array {
private:
    int arr[5];

public:
    void setElement(int index, int value) {
        if (index < 0 || index >= 5) {
            throw IndexOutOfBoundsException();
        }
        arr[index] = value;
    }
    int getElement(int index) {
        if (index < 0 || index >= 5) {
            throw IndexOutOfBoundsException();
        }
        return arr[index];
    }
};

int main() {
    Array myArray;

    try {
        myArray.setElement(2, 10);
        myArray.setElement(6, 20);
    }

    catch (const IndexOutOfBoundsException& e) {
        cout << e.what() << endl;
    }

    try {
        cout << "Element at index 2: " << myArray.getElement(2) << endl;
        cout << "Element at index 6: " << myArray.getElement(6) << endl;
    }
}

```

```
    }  
    catch (const IndexOutOfBoundsException& e) {  
        cout << e.what() << endl;  
    }  
    return 0;  
}
```

Output –

```
Error: Index is out of bounds!  
Element at index 2: 10  
Error: Index is out of bounds!
```